RPG IV Mise en œuvre

Structures de contrôle

Généralités

- Les structures de contrôle permettent d'exécuter des instructions différentes
  - En fonction de conditions
  - Dynamique : avec les données durant l'exécution
- Chaque test effectué est une valeur booléenne
  - Un test est vrai ou faux
  - Cette valeur résulte de l'évaluation d'une expression
  - Exemple

```
IF (a=b) ;
IF (a='Mardi');
IF (*in03 = *on );
```

Tests

Tests logiques

- Voir le chapitre ‘Arithmétiques’
- Rappel des opérateurs utilisables

Opérateur	Signification
=	Egal
<	Inférieur
>	Supérieur
<=	Inférieur ou égal
>=	Supérieur ou égal
<>	Différent
NOT, AND, OR	Non, Et, Ou
()	Parenthèses pour les priorités

Tests

- Les tests peuvent être réalisés par les codes opérations suivants

Opérateur	Signification
IF	Test simple avec alternative
SELECT	Test avec valeur multiples
CABxx	Comparaison et débranchement vers un tag
CASxx	Comparaison et débranchement vers une sous-routine

IF

- Syntaxe style RPG IV
  - IF condition

    Instructions condition vraie
  - ELSE

    Instructions condition fausse
  - END/ENDIF
- Règle
  - Les instructions sont facultatives
  - ELSE est facultatif
  - END ou ENDIF est obligatoire

    En format libre, seul ENDIF est autorisé
  - Dans cette syntaxe, le IF s’écrit en format long (facteur 2 étendu)

IF - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++++++++++++

      * Test simple
     C                   IF        NOMCLI=*blanks
     C                   EVAL      NUMCLI=0
     C                   ENDIF
      * Test avec alternative
     C                   IF        ZONE1='ABC'
     C                   MOVE      *BLANK        ZONE2
     C                   ELSE
     C                   MOVE      *ALL'*'       ZONE2
     C                   ENDIF
      * Test complexe avec alternative. Instructions multiples
     C                   IF        (Cpt=0 AND DateTrt<>%date) OR *in05
     C                   EVAL      NextTrt='SORTIR'
     C                   ELSE
     C                   EVAL      NextTrt='SUITE'
     C                   EVAL      Cpt=Cpt+1
     C                   ENDIF

      /free
       // Test simple avec alternative
       IF Zonec = 'XZER';
          Zoned = *blanks;
       Else ;
          Zoned = 'AZER';
       Endif;
      /end-free
```

IFxx

- Syntaxe style RPG/400
  - Facteur 1 IFxx Facteur 2
  - On peut faire des conditions complexes avec ANDxx et ORxx
- xx peut prendre la valeur
  - EQ Egal
  - NE Non égal (différent)
  - GT Supérieur
  - GE Supérieur ou égal
  - LT Inférieur
  - LE Inférieur ou égal

IFxx

- Règle
  - Format fixe non étendu
  - Réalise une comparaison entre facteur 1 et facteur 2
  - Existe pour la compatibilité avec RPG/400, n’est pas recommandé en RPG IV
    - Peu lisible pour les conditions complexes
    - Nécessite plus de lignes

IFxx - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtFactor2+++++++Result++++++++Len++D+HiLoEq

      * Test simple
     C     NOMCLI         IFEQ      *blanks
     C                    EVAL      NUMCLI=0
     C                    ENDIF

      * Test avec alternative
     C     ZONE1          IFEQ      'ABC'
     C                    MOVE      *BLANK        ZONE2
     C                    ELSE
     C                    MOVE      *ALL'*'       ZONE2
     C                    ENDIF

      * Test complexe avec alternative. Instructions multiples
      * Test : si (Cpt=0 AND DateTrt<>%date) OR *in05
     C                    EVAL      DateJour = %date
     C     *in05          IFEQ      *ON
     C     Cpt            OREQ      0
     C     DateTrt        ANDEQ     DateJour
     C                    EVAL      NextTrt='SORTIR'
     C                    ELSE
     C                    EVAL      NextTrt='SUITE'
     C                    EVAL      Cpt=Cpt+1
     C                    ENDIF
```

IF imbriqués

- Pour les tests imbriqués
  - IF condition

    Instructions condition vraie
  - ELSEIF autre condition

    Instructions condition vraie
  - ELSEIF autre condition

    Instructions condition vraie
  - ELSE

    Instructions par défaut
  - END/ENDIF
- Règle
  - ELSE est facultatif
  - END ou ENDIF est obligatoire
  - ELSEIF s’écrit en format long (facteur 2 étendu)

IF imbriqués - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++

     * IF imbriqués par ELSEIF
    C                   IF        Oper='INIT'
    C                   EVAL      Cpt=0
    C                   ELSEIF    Oper='PLUS'
    C                   EVAL      Cpt=Cpt+1
    C                   ELSEIF    Oper='MOINS'
    C                   EVAL      Cpt=Cpt-1
    C                   ELSE
    C                   EVAL      Cpt=*hival
    C                   ENDIF

     * Est équivalent à
    C                   IF        Oper='INIT'
    C                   EVAL      Cpt=0
    C                   ELSE
    C                   IF        Oper='PLUS'
    C                   EVAL      Cpt=Cpt+1
    C                   ELSE
    C                   IF        Oper='MOINS'
    C                   EVAL      Cpt=Cpt-1
    C                   ELSE
    C                   EVAL      Cpt=*hival
    C                   ENDIF
    C                   ENDIF
    C                   ENDIF
```

SELECT

- Pour les choix multiples
  - Evite d'imbriquer des IF
- Syntaxe
  - SELECT
  - WHEN condition

    Instructions condition vraie
  - WHEN autre condition

    Instructions condition vraie
  - OTHER

    Instructions par défaut
  - END/ENDSL

SELECT

- Règles
  - De 0 à n clauses WHERE
  - OTHER est facultatif
  - END ou ENDSL obligatoire
    - En format libre, seul ENDSL est autorisé
  - Dès qu'une clause WHEN est exécutée, le programme continue après END/ENDSL
  - Les conditions des clauses WHEN sont indépendantes les unes des autres

SELECT - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++

     C                   SELECT
     C                   WHEN      Oper='INIT'
     C                   EVAL      Cpt=0
     C                   WHEN      Oper='PLUS'
     C                   EVAL      Cpt=Cpt+1
     C                   WHEN      Oper='MOINS'
     C                   EVAL      Cpt=Cpt-1
     C                   WHEN      Oper='SUITE' AND
     C                             (*IN05=*ON OR Sortie='NON')
     C                   EVAL      Cpt=Cpt-1
     C                   OTHER
     C                   EVAL      CptT=*hival
     C                   ENDSL
```

SELECT avec comparaison

- Syntaxe style RPG/400
  - Pour compatibilité
- Même principe qu'avec le IFxx
  - WHENxx
  - On peut utiliser ORxx et ANDxx
- Règle
  - Format fixe
  - Effectuer une comparaison entre facteur1 et facteur2

SELECT et comparaison - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2+++++++++++++++++++++++++++++
.....CL0N01Factor1+++++++Opcode&ExtFactor2+++++++Result++++++++Len++D+HiLoEq....

     C                   SELECT
     C     Oper          WHENEQ    'INIT'
     C                   EVAL      Cpt=0
     C     Oper          WHENEQ    'PLUS'
     C                   EVAL      Cpt=Cpt+1
     C     Oper          WHENEQ    'MOINS'
     C                   EVAL      Cpt=Cpt-1
     C     Oper          WHENEQ    'SUITE'
     C     *IN05         ANDEQ     *ON
     C     sortie        OREQ      'NON'
     C                   EVAL      Cpt=Cpt-1
     C                   OTHER
     C                   EVAL      Cpt=*hival
     C                   ENDSL
```

CABxx

- Syntaxe style RPG/400
  - Pour compatibilité
- Comparaison et débranchement vers une étiquette
  - Est équivalent à un test IF suivi d'un GOTO
- Syntaxe
  - Facteur1 CABxx Facteur2 Etiquette
- Règles
  - De 1 à n clauses CABxx
  - Pas de début ni de fin de groupe
  - Effectue une comparaison entre facteur1 et facteur2. Débranche vers l'étiquette si la comparaison est vraie
  - Syntaxe fixe uniquement

CABxx - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++++++++++++++++++
.....CL0N01Factor1+++++++Opcode&ExtFactor2+++++++Result++++++++Len++D+HiLoEq....

     C                   EVAL      OPER=SAISIE
      * Débranchement vers le traitement nécessaire
     C     OPER          CABEQ     'INIT'        TAG1
     C     OPER          CABEQ     'PLUS'        TAG2
     C     OPER          CABEQ     'MOINS'       TAG3

      * Etiquettes :
     C     TAG1          TAG
     C                   EVAL      CPT=0
     C     TAG2          TAG
     C                   EVAL      CPT=CPT+1
     C     TAG3          TAG
     C                   EVAL      CPT=CPT-1
```

CASxx

- Syntaxe style RPG/400
  - Pour compatibilité
- Comparaison et débranchement vers une sous-routine
  - Est équivalent à un test IF suivi d'un EXSR
- Syntaxe
  - Facteur1 CASxx Facteur2 Sous-routine
  - END/ENDCS

CASxx

- Règles
  - De 1 à n clauses CASxx
  - END ou ENDCS obligatoire
  - Pas de début de groupe
  - Effectue une comparaison entre facteur1 et facteur2. Débranche vers la sous-routine si la comparaison est vraie
  - Syntaxe fixe uniquement

CASxx - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++++++++++++++++++
.....CL0N01Factor1+++++++Opcode&ExtFactor2+++++++Result++++++++Len++D+HiLoEq....

     * Débranchement vers la sous-routine
     C     OPER          CASEQ     'INIT'        SSR1
     C     OPER          CASEQ     'PLUS'        SSR2
     C     OPER          CASEQ     'MOINS'       SSR3
     C                   ENDCS


     * Sous-routine 1
     C     SSR1          BEGSR
     C                   EVAL      CPT=0
     C                   ENDSR
     * Sous-routine 2
     C     SSR2          BEGSR
     C                   EVAL      CPT=CPT+1
     C                   ENDSR
     * Sous-routine 3
     C     SSR3          BEGSR
     C                   EVAL      CPT=CPT-1
     C                   ENDSR
```

Boucles

Boucles

- Codes opérations pour les boucles

Opérateur	Signification
DO	Boucle n fois
FOR	Boucle n fois
DOU	Boucle tant que la condition est fausse
DOW	Boucle tant que la condition est vraie

- Code

Opérateur	Signification
ITER	Force l'itération suivante
LEAVE	Quitter la boucle

DO

- Syntaxe style RPG/400
  - Pour compatibilité
- Syntaxe avec index
  - Valeur_départ DO Valeur_limite Index

    Instructions
  - END/ENDDO

DO

- Syntaxe sans index
  - DO Nb_iterations

    Instructions
  - END/ENDDO
- Règles
  - END ou ENDDO obligatoire
  - La boucle est exécutée

    Syntaxe 1 : Tant que Valuer_départ <= Valeur_Limite

    La valeur courante est placée dans Index

    Syntaxe 2 : du nombre contenu dans Nb_Iterations
  - Syntaxe fixe uniquement

DO - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++++++++++++++++++
.....CL0N01Factor1+++++++Opcode&ExtFactor2+++++++Result++++++++Len++D+HiLoEq....

     D depart          s              3s 0 inz( 0 )
     D stop            s              3s 0 inz( 10 )
     D index           s              3s 0 inz
     D msg             s             20a   inz


      * Boucle 3 fois
     C                   DO        3
      * Faire quelque chose
     C     '...DO...'    DSPLY
     C                   ENDDO


      * Boucle tant que depart <= stop
     C     depart        DO        stop          index
      * Faire quelque chose
     C                   EVAL      msg = '...DO... : ' +
     C                             %char( index)
     C     msg           DSPLY
     C                   ENDDO
```

FOR

- Syntaxe style RPG IV
  - FOR Index = départ TO/DOWNTO stop BY incrément

    Instructions
  - END/ENDFOR

FOR

- Règles
  - END ou ENDFOR obligatoire
  - BY facultatif

    Par défaut, l'incrément est +1
  - TO/DOWNTO et BY n'ont pas d'ordre

    FOR I = 1 TO 10 BY 2

    FOR I = 1 BY 2 TO 10
  - Format fixe étendu et format libre
  - Boucle tant que

    Index <= départ (TO)

    Index >= départ (DOWNTO)

FOR - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++++++++++++++++++

     D index           s              3s 0 inz
     D msg             s             20a   inz('Texte à afficher')
     D msgImp          s             20a   inz
      * Boucle 3 fois
     C                   FOR       index = 1 TO 3
     C     '...FOR...'   DSPLY
     C                   ENDFOR
      * boucle du nombre de caractères contenus dans msg, sans les blancs
     C                   FOR       index = 1 TO %len( %trim( msg ) ) BY 2
      * affiche chaque caractère impaire :
     C                   EVAL      msgImp = %trimr( msgImp ) +
     C                                      %subst( msg : index : 1 )
     C                   ENDFOR
     C     msgImp        DSPLY

      /free
       FOR index = 1 TO 3 ; // Boucle 3 fois
          DSPLY '...FOR...' ;
       ENDFOR;
       // boucle du nombre de caractères contenus dans msg, sans les blancs
       FOR index = 1 TO %len( %trim( msg ) ) BY 2 ;
          // affiche chaque caractère impair :
          msgImp = %trimr( msgImp ) + %subst( msg : index : 1 ) ;
       ENDFOR;
       DSPLY msgImp ; // Affiche : Txeàafce
```

DOU

- Syntaxe
  - DOU condition

    Instructions
  - END/ENDDO
- Règles
  - END ou ENDDO obligatoire
  - La boucle est exécutée jusqu’à ce que la condition soit vraie
  - Format fixe étendu et format libre
  - La condition est totalement libre

DOU - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++++++++++++++++++

     * Boucle tant que l'utilisateur n'appuie pas sur F3
    C                   DOU       *inkc
    C                   EXFMT     ecran
     * traiter la saisie
    C                   ENDDO

     * boucle du nombre de caractères contenus dans msg, sans les blancs
    C                   DOU       index > %len( %trim( msg ) )
    C                   EVAL      index = index + 1
    C                   EVAL      msgImp = %trimr( msgImp ) +
    C                                       %subst( msg : index : 1 )
    C                   ENDDO
    C     msgImp        DSPLY

     /free
      // Boucle tant que l'utilisateur n'appuie pas sur F3
      DOU *inkc ;
         EXFMT ecran ;
      // traiter la saisie
      ENDDO ;

      // boucle du nombre de caractères contenus dans msg, sans les blancs
      DOU index > %len( %trim( msg ) ) ;
         index = index + 1 ;
         msgImp = %trimr( msgImp ) + %subst( msg : index : 1 ) ;
      ENDDO ;
      DSPLY msgImp ; // Affiche : Txeàafce
```

DOW

- Syntaxe
  - DOW condition

    Instructions
  - END/ENDDO
- Règles
  - END ou ENDDO obligatoire
  - La boucle est exécutée tant que la condition est vraie
  - Format fixe étendu et format libre
  - La condition est totalement libre

DOW - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++++++++++++++++++

      * Boucle tant que l'utilisateur n'appuie pas sur F3
     C                   DOW       not *inkc
     C                   EXFMT     ecran
      * traiter la saisie
     C                   ENDDO

      * boucle du nombre de caractères contenus dans msg, sans les blancs
     C                   DOW       index <= %len( %trim( msg ) )
     C                   EVAL      index = index + 1
     C                   EVAL      msgImp = %trimr( msgImp ) +
     C                                        %subst( msg : index : 1 )
     C                   ENDDO
     C     msgImp        DSPLY

      /free
       // Boucle tant que l'utilisateur n'appuie pas sur F3
       DOW not *inkc ;
          EXFMT ecran ;
       // traiter la saisie
       ENDDO ;

       // boucle du nombre de caractères contenus dans msg, sans les blancs
       DOW index <= %len( %trim( msg ) ) ;
          index = index + 1 ;
          msgImp = %trimr( msgImp ) + %subst( msg : index : 1 ) ;
       ENDDO ;
       DSPLY msgImp ; // Affiche : Txeàafce
```

DOWxx et DOUxx

- Syntaxe style RPG/400
  - Pour compatibilité
  - Facteur1 DOWxx/DOUxx Facteur2

    Instructions
  - END/ENDDO
- Comparaison et boucle

DOWxx et DOUxx

- Règles
  - END ou ENDDO obligatoire
  - DOUxx : La boucle est exécutée jusqu’à ce que la condition soit vraie
  - DOWxx : La boucle est exécutée tant que la condition est vraie
  - ANDxx et ORxx sont utilisables
  - Format fixe uniquement

DOUxx et DOWxx - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtFactor2+++++++Result++++++++Len++D+HiLoEq....

      * Boucle tant que l'utilisateur n'appuie pas sur F3
     C     *INKC          DOW       *OFF
     C                    EXFMT     ecran
      * traiter la saisie
     C                    ENDDO


      * Jusqu'à ce que le n°client soit 0, ou le nom à blanc, ou une erreur
     C     NUMCLI         DOU       0
     C     NOMCLI         OREQ      *blanks
     C     WSTATUS        OREQ      'ANO'
     C     CLE            CHAIN(E)  CLIENTS
     C                    IF        %error
     C                    EVAL      WSTATUS='ANO'
     C                    ELSE
      * Action sur le client
     C                    ENDIF
```

LEAVE et ITER

- ITER (itération suivante)
  - Passe directement au ENDDO/ENDFOR
  - La condition est retestée
- LEAVE
  - Sortie immédiate de la boucle : le contrôle passe à la première instruction après ENDDO/ENDFOR
- Règles
  - Aucun mot-clé
  - S’utilise dans n’importe quelle boucle

LEAVE et ITER - Exemple

```
.....CL0N01Factor1+++++++Opcode&ExtExtended-factor2++++++++++++++++++++++++++++

     C                   DOW       ZONEA <> 'FIN'
      * Si erreur sortie immédiate
     C                   IF        *IN99 = *ON
     C                   LEAVE
     C                   ENDIF
     C                   EVAL      CPT = CPT + 1
      * Itération suivante si nécessaire
     C                   IF        CPT < 10
     C                   ITER
     C                   ENDIF
      * Traitement de la demande
     C                   EVAL      PRIX = MU * ( 1 + TXTVA )
      * traiter la saisie
     C                   ENDDO

      /free
       DOW ZONEA <> 'FIN' ;
          IF *IN99 = *ON ; // Si erreur, sortie immédiate
             LEAVE ;
          ENDIF;
          CPT = CPT + 1 ;
          IF CPT < 10 ;    // itération suivante ?
             ITER ;
          ENDIF;
          PRIX = MU * ( 1 + TXTVA ) ;  // Prix de l'article
       ENDDO ;
```